

Retorno de la información P1

- Retorno de la información => proceso de retornar documentos (que suelen estar en lenguaje natural no estructurado) a partir de una colección de documentos en respuesta a una consulta.
- La información no estructurada:
 - No tiene un modelo formal bien definido
 - Se basa en la comprensión del lenguaje natural
 - Se almacena en una variedad amplia de formatos estándares
- Los sistemas de retorno de información (SRI) típicamente permiten expresiones de consulta formadas usando palabras clave y conectivos proposicionales.
- Ejemplos de lenguajes de consulta:
 - **Con frases:** se utiliza una secuencia de palabras como una frase.
 - **Con palabras clave:** se usan palabras clave, asumiendo "AND" entre ellas.
 - **Booleanas:** combinan términos con operadores booleanos (AND, OR, NOT).
 - **Con expresiones regulares:** emplean patrones específicos para la búsqueda.
 - **De proximidad:** especifican la cercanía entre términos, usando operadores como "NEAR" o "WITHIN/10".
 - **En lenguaje natural:** se introduce el texto como una oración o pregunta, comprendiendo estructura y significado.
 - **Google Search:** permite buscar en lenguaje natural y usa operadores como ``"`` para frases exactas, `OR`, `-` para excluir términos, `site::` para restringir el sitio, y `intitle::` para buscar en títulos específicos.

Resultados de una consulta

- Los resultados de una búsqueda pueden ser una lista de identificadores de documentos y también algunas piezas de texto. Los documentos suelen retornar en orden decreciente de puntaje de relevancia (usualmente se retornan unos pocos documentos del resultado y no todos).
- La relevancia se basa en factores como:
 - o Frecuencia de términos: frecuencia de ocurrencia de término de una consulta en un documento.(TF)
 - o Frecuencia inversa de documentos: Si la palabra ocurre en menos documentos se le da más importancia a la palabra.(IDF)
 - o Enlaces a documentos: Si hay más enlaces a un documento, el documento es más importante.
- La mayoría de los sistemas de retorno de la información (SRI) agregan:
 - o Las palabras que ocurren en el título, lista de autores, títulos se les da más importancia.
 - o Las palabras cuya primera ocurrencia es tarde en el documento se les da poca importancia.
 - o Proximidad: si las palabras de una consulta aparecen cerca entre sí en el documento, el documento tiene más importancia que si las palabras ocurren bien lejos unas de otras.

Enfoques estadísticos

- Un SRI al procesar una consulta no accede directamente a los documentos, sino que se usa una representación de cada documento (el cual resume lo que contiene y es relevante para el procesamiento de consultas).
- En un enfoque estadístico los documentos son analizados y descompuestos en piezas que pueden ser palabras, frases, n-gramas y cada pieza se la cuenta, pesa y mide para determinar su relevancia e importancia.

o Dada una consulta se comparan los términos de la consulta con las piezas para determinar un grado potencial de correspondencia y para producir una lista ordenada de documentos resultantes.

Modelo Booleano :

- Representa los documentos como un conjunto de términos.
- Utiliza consultas booleanas y devuelve sólo coincidencias exactas.
- Un documento es relevante o no, sin orden de relevancia entre resultados.

Modelo de Espacio Vectorial :

- Representa cada documento como un vector en un espacio de n dimensiones, donde cada dimensión es un término.
- El peso de cada término en el vector puede basarse en su presencia, frecuencia o en el cálculo de TF-IDF.
- La relevancia de un documento se determina usando una función de similitud de vectores, permitiendo ordenar los resultados.
- Se puede establecer la medida de relevancia de un documento con respecto a una consulta. Una consulta también se representa como un vector de términos. El vector de una consulta es comparado con los vectores de los documentos para estimación de similitud/relevancia.
- Los términos tienen peso en los vectores. Una opción es que el peso sea la frecuencia de cada término en el documento/consulta y otra posibilidad es usar TF-IDF (frecuencia de término-frecuencia inversa de documento) que permite evaluar la importancia de una palabra en una colección de documentos.
- La relevancia de un documento se mide a través de una función de similitud de vectores, lo que permite ordenar los resultados. Tanto los documentos como las consultas se representan como vectores de términos, y se comparan entre sí para estimar su relevancia. Los términos en estos vectores tienen un peso, que puede basarse en la frecuencia de cada término o en el método TF-IDF, que evalúa la importancia de una palabra en relación con una colección de documentos. (resumen de lo anterior)
- Idea de TF-IDF: términos que capturan la esencia de un documento aparecen frecuentemente en el mismo; pero para que un término sea bueno para discriminar un documento de los demás, entonces debe ocurrir en unos pocos documentos de la colección

Modelo de espacio vectorial • Sea término i y documento Dj • $TF_{ij} = f_{ij} / \sum_{i=1 \text{ to } V } f_{ij}$ • $IDF_i = \log(N / n_i)$ - o Tf_{ij} es la frecuencia del término i en el documento Dj normalizada. - o f_{ij} es la cantidad de ocurrencias del término i en el documento Dj. - o N es el número de documentos en la colección. - o n_i es el número de documentos donde el término i ocurre.	Modelo de espacio vectorial • Se puede calcular la relevancia de un documento Dj con respecto a una consulta Q de la siguiente manera: $rel(D_j, Q) = \sum_{i \in Q} TF_{ij} \times IDF_i$ • Observación: si la consulta dice que el término i no debe considerarse, entonces ese término no debería estar en la fórmula.
Modelo de espacio vectorial • Se usa función de similitud de vectores. • La función del coseno del ángulo entre los vectores de la consulta y el documento se usa frecuentemente para estimar la similitud. $cosine(d_j, q) = \frac{\langle d_j, q \rangle}{\ d_j\ \times \ q\ } = \frac{\sum_{i=1}^{ V } w_{ij} \times w_{iq}}{\sqrt{\sum_{i=1}^{ V } w_{ij}^2} \times \sqrt{\sum_{i=1}^{ V } w_{iq}^2}}$ • Donde d_j es el vector del documento j, q es el vector de la consulta. • w_{ij} es el peso del término i en el documento j, w_{iq} es el peso del término i en el vector de la consulta q. • V es el número de dimensiones en el vector.	Modelo de espacio vectorial • Interpretación del resultado de la función de similitud de vectores: - o Si el resultado es cercano a 1, significa que los vectores son muy similares. - o Si el resultado es cercano a 0, los vectores no tienen casi nada en común. - o Si el resultado es -1, entonces los vectores son opuestos. • La función de similitud de vectores se puede usar para calcular la similitud entre el vector de una consulta y el vector de un documento. - o Este valor puede usarse también como medida de relevancia de un documento con respecto a una consulta.

Selección de términos relevantes de un documento:

Stopwords : Son palabras comunes que aparecen en la mayoría de los documentos y no aportan relevancia, por lo que deben eliminarse de las consultas.

Variantes de términos : Algunos términos tienen múltiples formas (computer y computing). Reducir palabras a una raíz o base común (algoritmo Martin Porter) mejora eficiencia.

Sinónimos : Usar un término por concepto ayuda a evitar redundancia, aunque es complejo por los múltiples significados que pueden tener en distintos contextos.

WordNet organiza palabras en conjuntos de sinónimos llamados *synsets*, categorizados por tipo gramatical (sustantivos, verbos, adjetivos, adverbios) y organizados jerárquicamente. Los sinónimos en Sistemas de Recuperación de Información (SRI) mejoran la precisión, cobertura y experiencia del usuario al facilitar la comprensión de la intención de búsqueda, ampliando el alcance sin necesidad de coincidencia exacta de términos.

Recopilación de entidades: Permite extraer hechos, eventos y nombres, entre otros, usando reglas, expresiones regulares y patrones lingüísticos, incluyendo sinónimos. Esto mejora la precisión en las búsquedas y agrupa información relevante aunque se usen términos distintos para referirse a las mismas entidades. Esto es muy útil para:

- **Mejorar la precisión en las búsquedas**: Usando esto, un SRI puede entender mejor las búsquedas del usuario y entregar resultados más relevantes.
- **Agrupar información relacionada**: facilita la identificación de documentos que mencionan las mismas entidades, aunque usen términos diferentes.

Índices invertidos

- ¿Cómo buscar las ocurrencias de un término en los documentos de una colección?
- Solución 1: se puede escanear secuencialmente cada documento.
- Pero si las colecciones de documentos son grandes ...
- Solución 2: Se usa una estructura de datos de índice invertido. Esta estructura puede tener para cada término los identificadores de los documentos donde aparece el término y también las posiciones en el documento donde aparece el término.

• Implementación de índices invertidos:

La lista invertida de un término puede ocupar múltiples bloques de disco. Para optimizar el acceso, esta lista puede almacenarse en bloques consecutivos, y un índice B + se usa para mapear cada término a su lista invertida, mejorando la eficiencia de búsqueda.

• Construcción de índices invertidos:

1. Extraer el vocabulario (términos) de los documentos de la colección
2. Armar estadísticas para cada documento dependiendo del modelo usado.
3. Invertir el stream de documentos con sus términos en un stream de términos y sus documentos. Aquí se puede agregar información adicional como frecuencias de términos, posiciones de términos y pesos de términos.

• Búsqueda de documentos relevantes a partir de consulta usando índice invertido :

1. Búsqueda de vocabulario: Cada término de la consulta se busca en el vocabulario. Los términos se pueden ordenar lexicográficamente para mejorar la eficiencia.
2. Retorno de la información de documentos: se retorna la información del documento para cada término.
3. Manipulación de la información retornada: la información de cada documento es procesada para incorporar las distintas formas de lógica de consulta.

- **S/lo visto en clases.**

- 1 Encontrar el vocabulario de la consulta
- 2 Buscar el término de la consulta en índice invertido
- 3 Calcular S_i documentos con términos
- 4 Evaluar la expresión de consulta usando los operadores asociados a los conectivos bool
- 5 Cálculo del vector de consulta
- 6 Calcular relevancia de cada documento usando similitud de vectores
- 7 Ordenar por relevancia de documentos

- **Tratamiento de consultas booleanas:**

La consulta involucra n términos. Si es conjunto de identificadores de documentos donde aparece término i ($i \in \{1, \dots, n\}$).

- o And: Los documentos deseados son: $S_1 \cap S_2 \cap \dots \cap S_n$.
- o Or: Los documentos deseados son: $S_1 \cup S_2 \cup \dots \cup S_n$.
- o Not: Sea t_i término i . Explicamos not t_i . Sea S el conjunto de todos los identificadores de documentos. Podemos eliminar los documentos que contienen el término i haciendo: $S - S_i$.
- En una alternativa para AND, se devuelven documentos con al menos uno de los términos, ordenados por relevancia.

- **Medición de la relevancia de los resultados de una consulta**

Los Sistemas de Recuperación de Información (SRI) solo permiten un retorno aproximado de resultados, lo cual puede provocar falsos negativos (documentos relevantes no retornados) y falsos positivos (documentos irrelevantes retornados). Los documentos relevantes retornados se llaman verdaderos positivos y los resultados irrelevantes no retornados se llaman verdaderos negativos. Las métricas clave para evaluar su desempeño son:

- Precisión: mide el porcentaje de documentos relevantes en los resultados.
- Cobertura: mide el porcentaje de documentos relevantes que se recuperan respecto al total en la base de datos.

Mejorar la cobertura puede reducir la precisión. Definir relevancia es complejo y requiere entender el lenguaje natural y el propósito de la consulta; por ello, el proceso suele incluir la creación manual de consultas y el etiquetado de documentos.

- **Problema:** Alta precisión es lograda casi siempre a expensas de cobertura y recíprocamente.
- La medición **F-score** es usada como una medida que combina precisión y cobertura para comparar distintos conjuntos de resultados (es el promedio armónico de los dos números).

$$F = \frac{2pr}{p+r} \quad \text{equivalentemente:} \quad F = \frac{2}{\frac{1}{p} + \frac{1}{r}}$$

- F tiende a ser cercano al más pequeño de p y r . Para F alto tanto r como p deben ser altos.

Lucene

Es una máquina de búsqueda e indexado ampliamente utilizada en la industria y academia, capaz de procesar grandes colecciones de documentos. Los documentos en Lucene se organizan en campos que pueden ser binarios, numéricos o de texto (tokenizado o no). Lucene ofrece una API de consultas que permite realizar búsquedas con operadores booleanos, expresiones regulares o de proximidad, y retorna documentos ordenados por

relevancia mediante una variante de TF-IDF. Además, emplea el modelo de espacio vectorial para el procesamiento de consultas.

Retorno de la información P2

Las **máquinas de búsqueda** procesan los sitios web y documentos de la web, manteniendo un repositorio actualizado de páginas organizadas mediante índices invertidos. Devuelven los resultados más relevantes, ordenados por relevancia. Las **máquinas de búsqueda verticales**, en cambio, se enfocan en temas específicos y cubren sólo una colección particular de documentos en la web.

• **Rastreadores web**

El problema en la web es que no se tiene una colección predefinida de documentos para buscar, por lo que es necesario construirla utilizando hiperenlaces. La solución consiste en utilizar **rastreadores web**, programas que localizan y recopilan información siguiendo los hiperenlaces de documentos conocidos para encontrar otros documentos, comenzando desde un conjunto inicial/semilla de documentos. Sin embargo, este proceso puede ser lento debido a la inmensidad de la web (puede llevar meses o años). Para agilizar el rastreo, se implementan múltiples procesos en paralelo en diferentes máquinas, almacenando los enlaces a rastrear en una base de datos y añadiendo nuevos enlaces encontrados para futuros rastreos.

Ejemplo de 3 problemas que debe resolver un rastreador web:

Acceso Restringido: Los rastreadores pueden encontrar dificultades si el contenido del sitio está protegido por contraseña o si se redirige a un inicio de sesión.

Manejo de contenido dinámico y cambiante: Las páginas web pueden actualizarse frecuentemente e incluir contenido dinámico. Un rastreador debe saber cómo capturar este tipo de contenido y decidir cuándo y con qué frecuencia volver a visitar páginas para mantener la información actualizada sin desperdiciar recursos.

Problemas de Redireccionamiento: Los sitios que utilizan múltiples redireccionamientos pueden generar complicaciones si algunos de estos redireccionamientos fallan o no están configurados correctamente

• **Indexado**

Los documentos recolectados por los rastreadores son procesados por un sistema de indexado, que puede descartar documentos o almacenar copias en caché. Sin embargo, un desafío es que el indexado consume tiempo, y las máquinas de búsqueda no pueden detenerse para responder consultas. Solución: ejecutar el indexado en varias máquinas, creando una nueva copia del índice sin modificar el anterior. Mientras tanto, se utiliza el índice antiguo para atender consultas. Una vez finalizado el rastreo y justo antes de iniciar un nuevo indexado, el índice nuevo se convierte en el índice antiguo.

• **Búsquedas**

Para manejar el problema de demasiadas consultas simultáneas en una máquina de búsqueda, la solución es utilizar múltiples máquinas para responderlas. Esto permite mantener los índices en memoria y enrutar las consultas a diferentes máquinas, lo que facilita el balanceo de carga.

• **Relevancia de documentos en la web**

Las primeras máquinas de búsqueda en la web clasificaban las respuestas únicamente en función de la relevancia basada en TF-IDF, lo que presentaba problemas. Este enfoque podría generar un gran número de documentos relevantes, pero muchas páginas de interés para los usuarios podían tener baja frecuencia de términos y, por ende, un bajo valor TF-IDF. Además, existía el riesgo de que páginas spam manipularan palabras clave para aparecer en los resultados, y los usuarios preferían sitios populares.

Para solucionar esto, se propuso utilizar la popularidad del sitio web, medida por su tráfico, como un criterio de clasificación. Sin embargo, obtener datos precisos sobre la popularidad era complicado, ya que requería la cooperación de los sitios y algunos podrían falsear esta información.

Una solución más refinada consiste en combinar las medidas tradicionales de relevancia, como TF-IDF, con la popularidad del sitio, para crear una evaluación global de la relevancia de una página en relación con una consulta. De este modo, las páginas con mayor relevancia se pueden presentar como las principales respuestas a la consulta.

• **Popularidad de sitios web**

Para medir la popularidad de un sitio web, se proponen varias ideas:

1. Archivos de marcadores de usuarios: Se podría usar la cantidad de veces que un sitio aparece en los marcadores de los usuarios para determinar su popularidad. Sin embargo, este enfoque tiene limitaciones, ya que los archivos de marcadores son generalmente privados y no accesibles públicamente.

2. Número de enlaces entrantes: Se sugiere utilizar la cantidad de enlaces a un sitio como medida de su prestigio. Contar enlaces desde otros sitios hacia el sitio en cuestión puede indicar su popularidad, aunque esta métrica se refiere más a sitios en general que a páginas individuales, dado que la mayoría de los enlaces apuntan a la página principal.

-Refinamiento de la métrica: Al calcular el prestigio basado en enlaces, se puede asignar un mayor peso a los enlaces provenientes de sitios de mayor prestigio.

3. Tasa de selección de páginas: Las máquinas de búsqueda pueden rastrear qué proporción de usuarios elige una página en los resultados de búsqueda. Esta información también puede utilizarse como un indicador de la popularidad del sitio.

• **PageRank**

• Algoritmo de ordenamiento PageRank: Algoritmo definido por Google como una medida de popularidad de una página basada en la popularidad de las páginas que la enlazan (viendo enlaces hacia fuera y dentro). Se considera a las páginas altamente enlazadas por otras páginas como más importantes que las páginas con menos enlaces hacia ellas.

o Supongamos que una persona navegando en la web realiza una caminata aleatoria de páginas web de la siguiente manera:

➤ El primer paso comienza en una página web aleatoria.

➤ En los pasos siguientes la persona hace una de las siguientes:

• Con una $p(\delta)$ la persona salta a una página web elegida aleatoriamente.,

• Con una $p(1 - \delta)$ la persona elige aleatoriamente uno de los enlaces hacia afuera de la página actual y sigue ese enlace.

o Entonces: PageRank => probabilidad de que esa persona visite la página en un determinado punto del tiempo.

• Las páginas apuntadas por páginas web con alto PageRank van a tener una mayor probabilidad de ser visitadas y entonces van a tener un mayor PageRank.

El algoritmo PageRank se basa en un sistema de ecuaciones lineales. Primero, se asigna un identificador a cada página web. Luego, se define la matriz de probabilidad de salto T , donde $T[i,j]$ representa la probabilidad de que un caminante aleatorio siga un enlace de la página i hacia la página j . Si todos los enlaces de i tienen la misma probabilidad de ser seguidos, entonces $T[i,j]=1/N_i$, siendo N_i el número de enlaces desde i .

- El PageRank de la página j puede definirse como:

$$P[j] = \delta/N + (1 - \delta) * \sum_{i=1}^N (T[i, j] * P[i])$$

- Donde δ es una constante entre 0 y 1 y N es el número de páginas.
- δ representa la probabilidad de un paso en la caminata aleatoria sea un salto aleatorio.

Para resolver el sistema, se usa un método iterativo: se inicializa cada $P[i]$ a $1/N$ y en cada paso se recalculan los valores de $P[i]$ en función de la iteración anterior. El proceso termina cuando el cambio máximo en los valores de $P[i]$ cae por debajo de un umbral definido.

• **Popularidad de sitios web**

- Problema: PageRank asigna una medida de popularidad que no considera los términos de la consulta.
- Solución: usar las palabras clave en el texto del áncora de los enlaces a una página para juzgar para qué tópicos la página es altamente relevante.
 - o La popularidad basada en texto de áncoras puede ser usada en combinación con otras medidas de popularidad y con TF-IDF para obtener un ranking de los resultados de una consulta.
- Implementación 1: Si se considera el texto de esas áncoras como parte de la página apuntada, entonces TF-IDF toma texto de áncoras en cuenta.
- Implementación 2: se computa una medida de popularidad usando solo páginas que contienen los términos de la consulta en lugar de computar popularidad usando todas las páginas web disponibles.
 - o Este enfoque es más costoso porque el computo del ranking de popularidad es dinámico ante una consulta, mientras que PageRank se computa estáticamente una vez y se reutiliza.

• **Relevancia de documentos en la web**

Para mejorar el ranking de resultados en los motores de búsqueda, los valores de TF-IDF de una página se deben combinar con métricas de popularidad como PageRank. Sin embargo, cómo se realiza esta combinación exacta es un secreto que muchas empresas guardan para evitar tanto la competencia como prácticas de spam. Una posible solución es usar una fórmula con pesos para cada factor. Estos pesos se determinan mediante un conjunto de entrenamiento de consultas, cuyo ranking ha sido fijado por humanos. A través de este conjunto, un algoritmo de aprendizaje automático puede ajustar los valores óptimos para esos parámetros.

• **Buscador de Google**

- Características principales del buscador de Google para la web:
- Rastreo: Google utiliza arañas web (Googlebot) que recorren la web para encontrar nuevas y actualizadas páginas.
- Indexación: Las páginas encontradas son almacenadas en el índice de Google, una base de datos masiva que contiene la copia de todas las páginas web visitadas por los rastreadores.
- Algoritmos de búsqueda: Google emplea complejos algoritmos para interpretar las consultas de los usuarios y encontrar los resultados más relevantes. Estos algoritmos consideran cientos de factores, incluyendo palabras clave, sinónimos y la ubicación del usuario.

- Relevancia de respuestas: Los algoritmos de Google no sólo encuentran coincidencias de palabras clave, sino que también intentan entender el contexto y la intención de la búsqueda para ofrecer los resultados más útiles y precisos.

- oRankings y PageRank: Google clasifica los resultados de búsqueda basándose en la autoridad y relevancia de las páginas, utilizando medidas como el PageRank, que evalúa la cantidad y calidad de los enlaces hacia una página.

- oPersonalización: Las búsquedas se personalizan en función de la historia del usuario, su ubicación y otros datos disponibles.

- oCalidad del contenido: Google valora el contenido original, de alta calidad y relevante, penalizando a sitios con prácticas de spam o contenido duplicado.

- oLenguaje de consulta: ya hablamos de ello en el archivo de filmas anterior.

- **Datos que contiene un enlace devuelto por el buscador de Google:**

1. Título de la página: El encabezado principal que resume de qué trata la página.
2. URL: La dirección web del sitio. Esto te da una idea de la fuente del contenido.
3. Extracto o snippet: Breve fragmento de texto que extrae de la página para mostrar una vista previa del contenido. Puede incluir las palabras clave de tu búsqueda resaltadas.
4. Breadcrumb: Una ruta de navegación que muestra dónde se encuentra la página dentro de la jerarquía del sitio web.
5. Fecha: Especialmente en artículos y noticias, se incluye la fecha de publicación o de la última actualización.

- **El buscador de Google ofrece más que solo enlaces como resultado de las búsquedas:**

- 1.Fragmentos destacados: Resúmenes directos de respuestas que se muestran en la parte superior de la página de resultados, a menudo en formato de párrafo.
- 2.Panel de conocimiento: Información detallada sobre un tema, persona, lugar o cosa, normalmente a la derecha de los resultados.
- 3.Videos: Recomendaciones de videos de plataformas como YouTube que son relevantes para tu consulta.
- 4.Imágenes: Una selección de imágenes relacionadas con tu búsqueda.
- 5.Preguntas relacionadas: Una lista de preguntas adicionales que otros usuarios han hecho sobre el mismo tema o uno similar, junto con respuestas breves que se pueden expandir.
- 6.Gráficos y estadísticas: En algunos casos, como para datos financieros o demográficos, Google puede mostrar gráficos interactivos.

Una introducción al procesamiento de lenguaje natural

- El procesamiento del lenguaje natural (PLN) es una subdisciplina de la inteligencia artificial que se ocupa de la comprensión, interpretación y generación del lenguaje humano por parte de las máquinas. Haciendo que las computadoras comprendan el texto y el habla de manera similar a como lo haría un ser humano

- **Componentes Clave:**

1. Análisis Sintáctico: Estudia la estructura gramatical de las oraciones, identificando partes del discurso (sustantivos, verbos, etc.) y su relación.
2. Análisis Semántico: Se enfoca en el significado de las palabras y frases en contexto, ayudando a las máquinas a entender el contenido.
3. Reconocimiento de Entidades Nombradas (NER): Identifica y clasifica entidades dentro del texto.
4. Generación de Lenguaje Natural (NLG): Permite a las máquinas generar texto coherente y relevante a partir de datos estructurados.
5. Análisis de Sentimientos: Evalúa el tono emocional del texto, determinando si es positivo, negativo o neutral.

- Aplicaciones Comunes: Asistentes Virtuales, Traducción Automática, Chatbots y Análisis de Opiniones

- La comprensión del lenguaje natural (NLU) y la generación de lenguaje natural (NLG) son subconjuntos importantes del PLN. Donde NLU está centrado en capacitar a las máquinas para comprender e interpretar el lenguaje humano de forma significativa y trata de entender el contexto, la intención y el sentimiento que hay detrás del lenguaje utilizado. Por otro lado, NLG se centra en la creación automática de narraciones escritas o habladas a partir de datos estructurados. Permite a las máquinas generar textos coherentes, contextualmente relevantes y parecidos al lenguaje humano. El NLG puede aplicarse en diversos ámbitos (generación de informes, respuestas de atención al cliente y creación de contenidos).

- Ejemplos de aplicaciones cotidianas de NLU: un asistente virtual, los chatbots, el análisis de sentimiento donde se analizan comentarios en redes sociales, reseñas de productos y encuestas, clasificación de correos electrónicos (ejemplo: si es spam o no)

- **Etapas de la comprensión del lenguaje natural**

1. **Preprocesamiento:** Consiste en limpiar y preparar los datos de texto para el análisis.
 - Contempla: tokenización (descomposición del texto en unidades más pequeñas), etiquetado de parte del discurso (identificación de funciones gramaticales de las palabras), análisis sintáctico (de la estructura gramatical de las frases).
2. **Análisis semántico:** se centra en comprender el significado de las palabras en su contexto.
 - Contempla: reconocimiento de entidades (identificación y clasificación de entidades clave en el texto), extracción de relaciones entre las diferentes entidades del texto.
3. **Comprensión del contexto:** Se analiza el contexto más amplio de una conversación o texto.
 - Contempla: evaluar el tema general y los matices que afectan a la interpretación, reconocer significados específicos del contexto que pueden cambiar en función de las interacciones anteriores o del texto circundante.
4. **Reconocimiento de intenciones:** Determina la intención del usuario tras un texto o discurso determinado.

- **Etapas de la generación del lenguaje natural**

1. **Determinación de contenido:** se decide qué información debe incluirse en el texto generado.
2. **Estructuración de documentos:** Se organiza la información de manera lógica.
3. **Planificación de frases:** el sistema planifica cómo expresar en frases el contenido seleccionado.
4. **Generación de expresiones de referencia:** se trata de crear expresiones que hagan referencia a entidades o conceptos mencionados en el texto.
5. **Realización:** Las frases planificadas se transforman en texto real.
6. **Post procesamiento (opcional):** se produce un refinamiento adicional luego de la generación inicial del texto.

- **Etapas del preprocesamiento**

1. **Limpieza de datos:** se eliminan elementos innecesarios del texto que pueden inferir con el análisis (se eliminan caracteres especiales, conversión a minúsculas y eliminación de etiquetas HTML si el texto viene de la web).
2. **Tokenización:** Se divide el texto en unidades más pequeñas llamadas tokens.
3. **Eliminación de stop words:** elimina palabras que no aportan significado relevante al análisis, como “y”, “el”, “de”.
4. **Stemming y lematización:**

- Stemming es un proceso que corta las palabras a su raíz, eliminando sufijos y prefijos. P.ej: “cocinando” se convierte en “cocin”.
 - Lematización: similar a stemming, pero devuelve la forma base o raíz de una palabra considerando su contexto gramatical. Ej: “mejores” se convierte en “bueno”.
5. Análisis morfológico: se centra en la estructura interna de las palabras, identificando sus componentes (raíces, sufijos).
 6. Part of speech tagging: Asigna etiquetas gramaticales a cada token (sustantivo, verbo, adjetivo, etc.)
 7. Conversión a formato estructurado: se transforma el texto preprocesado a un formato que pueda ser fácilmente analizado por algoritmos.

• **Etapas del Análisis Semántico**

1. Desambiguación de Palabras: Este proceso implica determinar el significado correcto de una palabra que puede tener múltiples significados (polisemia) dependiendo del contexto.
2. Reconocimiento de Entidades Nombradas (NER): Consiste en identificar y clasificar entidades clave dentro del texto.
3. Análisis de Relaciones: Identificar las relaciones entre las entidades dentro del texto.
4. Análisis de Sentimientos: Se utiliza para determinar el tono emocional del texto.
5. Comprensión Contextual: Implica analizar el contexto general en el que se presenta el texto, incluyendo el tema y las interacciones previas.
6. Construcción de Representaciones Semántica: Se refiere a la creación de modelos que representan el significado del texto en un formato estructurado que las máquinas pueden procesar.

El contexto

- En el marco del PLN, el término contexto se refiere a la información adicional que rodea una palabra, frase o conversación que ayuda a interpretar su significado y a entender la intención detrás de las interacciones.
 - Esto es crucial para desambiguar significados, mantener la coherencia en el diálogo y proporcionar respuestas relevantes.
1. Contexto Local: Entorno inmediato de una palabra o frase en una oración. Incluye las palabras que preceden y siguen a la unidad de texto en cuestión.
 2. Contexto Global: Abarca información que se extiende más allá de una sola oración o párrafo. Incluye el historial de interacciones anteriores.
 3. Contexto Conversacional: Se refiere a la dinámica del diálogo entre los participantes, incluyendo turnos de habla y respuestas anteriores.
 4. Contexto Situacional: Incluye factores externos que pueden influir en la interpretación del lenguaje (entorno físico, estado emocional del hablante o contexto cultural).
 5. Contexto Histórico: Se refiere al conocimiento acumulado a lo largo del tiempo sobre un tema o sobre las preferencias del usuario.
 6. Contexto Cultural: Incluye factores sociales y culturales que influyen en cómo se interpreta el lenguaje.

• La comprensión del contexto es fundamental para interpretar correctamente el lenguaje natural. Esto incluye el seguimiento del contexto conversacional (recordar interacciones pasadas y usarlas en respuestas futuras), el uso de memoria contextual en algunos sistemas NLU (donde se retiene información de las interacciones anteriores), y el contexto situacional (factores externos que influyen en la interpretación como ubicación, hora, o eventos actuales). Además, el análisis de sentimientos permite captar emociones en el lenguaje y ajustar las respuestas. La adaptación dinámica es clave para mejorar la interacción de humanos y máquinas.

• **Etapas de Comprensión del contexto**

1. **Análisis Sintáctico:** Implica descomponer una oración en sus componentes gramaticales, como sujeto, verbo y objeto, para entender su estructura.
2. **Desambiguación de Palabras:** Se refiere a la identificación del significado correcto de palabras que pueden tener múltiples significados dependiendo del contexto.
3. **Análisis de Relaciones:** Esta etapa se centra en entender cómo las diferentes entidades y conceptos dentro del texto están relacionados entre sí.
4. **Análisis Pragmático:** considera el contexto situacional, la intención del hablante y las implicaciones sociales. Permite interpretar correctamente el lenguaje en situaciones específicas, teniendo en cuenta factores como el tono, las convenciones conversacionales y las expectativas culturales.
5. **Resolución de Correferencia:** tarea de identificar a qué entidades se refieren los pronombres o expresiones en un texto. Permite al sistema recordar a quién o qué se refiere cuando se utilizan términos como "él", "ella" o "eso".
6. **Comprensión Contextual:** Implica analizar el contexto más amplio en el que se presenta el texto, incluyendo interacciones previas y el tema general de la conversación.
7. **Análisis de Sentimientos:** Determina el tono emocional del texto, identificando si es positivo, negativo o neutral.
8. **Adaptación al Contexto Cultural:** Considera las variaciones culturales y lingüísticas que pueden influir en la interpretación del lenguaje.

Ingeniería de prompts

- Un prompt es una instrucción o pregunta que se le proporciona al modelo basado en PLN para indicarle el tipo de respuesta que se espera.
- Prompt engineering, se refiere a la habilidad de diseñar y optimizar prompts. Esto no solo implica diseñar y refinar los prompts para obtener los resultados deseados de manera eficiente y precisa, sino también ajustar y probar diferentes enfoques en los modelos de PLN para mejorar su efectividad y precisión.
- El ajuste y prueba de diferentes enfoques dentro de prompt engineering, son las modificaciones y optimizaciones específicas para mejorar la respuesta del modelo:
 - o Ajustar los parámetros del modelo.
 - o Incorporar más datos de entrenamiento relevantes.
 - o Refinar la estructura del prompt para que el modelo entienda mejor la intención.
 - o Experimentar con diferentes formulaciones para ver cuál produce las mejores respuestas.
- Un buen ingeniero de prompts debe tener una mezcla de habilidades técnicas y creativas:
 - o Conocimiento en PLN: Entender cómo funcionan los modelos de PLN.
 - o Creatividad: Pensar para formular prompts efectivos.
 - o Análisis de datos: Evaluar y ajustar prompts basándose en resultados y métricas de rendimiento.
 - o Resolución de problemas: Identificar y solucionar problemas en la generación de respuestas.
 - o Comunicación clara: Formular prompts y objetivos de manera precisa y comprensible.
 - o Experimentación: Probar diferentes enfoques y ajustar parámetros para optimizar resultados.
 - o Comprensión de contexto: Captar la intención y contexto detrás de las preguntas para generar respuestas precisas.

Conceptos básicos

Los modelos de texto actuales procesan el lenguaje dividiéndolo en tokens, que son secuencias de caracteres tratadas como unidades. El uso de la API y la capacidad máxima de procesamiento de los modelos se miden en tokens. Estos modelos no comprenden el

texto como los humanos, pero pueden predecir y continuar frases con gran precisión al calcular las secuencias de palabras más probables. Funcionan como máquinas de probabilidad y lenguaje, y esta estructura permite detectar textos generados por IA analizando la probabilidad de cada palabra. Los textos de IA tienden a usar palabras más probables, mientras que los humanos emplean palabras menos previsibles.

• **El playground de OpenAI y sus parámetros**

El playground de OpenAI permite experimentar con los modelos GPT, seleccionando el modelo de lenguaje y ajustando parámetros clave como la longitud de salida y la temperatura. La temperatura controla la creatividad en las respuestas: una temperatura alta genera respuestas más diversas pero menos precisas, mientras que una baja las hace más predecibles y fiables. El parámetro TopP limita las opciones de texto, siendo que con TopP alto el modelo va a considerar más opciones, aunque tengan probabilidad baja produciendo respuestas más creativas, pero con mayor probabilidad que sean respuestas más irrelevantes o incluso incorrectas. Los parámetros frequency penalty y presence penalty reducen la repetición en el texto generado.

- **Frequency Penalty:** Este parámetro penaliza la aparición repetida de tokens (palabras o frases) en la respuesta.
- **Presence Penalty:** Este parámetro penaliza la aparición de cualquier token que ya haya sido utilizado en la respuesta.

• **Las diferentes partes de un prompt**

• Vamos a dividir un prompt en: Prompt = acción + modificadores + datos

• **Acciones:** Qué queremos que realice el modelo de texto. Normalmente son verbos y órdenes sencillas (escribe, resume, traduce, categoriza) o preguntas guiadas p.ej: ¿qué opinas sobre ...?. En un mismo prompt puede haber varias acciones.

• **Modificadores:** Elementos que agregamos a la acción para completar los resultados y dar forma a la respuesta del modelo. Cuantos más, más se va a moldear la respuesta del modelo de texto. Hay diferentes tipos de modificadores que se pueden usar en un prompt.

- **Tipo rol:** decirle a la IA la personalidad que adoptará. Usará unas relaciones entre tokens propias de ese rol o esa personalidad.

- **Asunto o tema:** sirve para constreñir el tema sobre el que queremos que la IA realice una acción.

- **Tipología de texto:** sirve para constreñir el tipo de texto

- **Modificador longitud de texto:** constriñe la cantidad de texto que queremos que responda el modelo.

- **Modificador audiencia objetivo:** constriñe el texto según a quién va dirigida la respuesta del modelo.

- **Modificador formato y estructura:** constriñe el formato y/o estructura del texto generado.

- **Modificador inserciones de texto:** sirve para localizar el punto donde queremos que la IA introduzca un determinado texto.

• **Datos:** Para que la IA incluya datos específicos en su respuesta, debemos dárselos directamente en el prompt o aparte, ya que solo puede usar la información que se le suministra en ese momento.

- **Standard prompt:** La IA no genera una cadena de pensamiento que le ayude a dar su respuesta. Se trata de una respuesta directa, sin generar el proceso paso a paso que le ha llevado a dar dicha respuesta.

- **Cadena de pensamiento - CoT :** el modelo de IA genera una cadena de pensamiento que le ayudará a dar su respuesta; respuesta generando el paso a paso.

Lo podemos hacer directamente; por ejemplo: poniendo al final del prompt la frase: resuelve el problema paso a paso o resuelve el problema generando todos los pasos y soluciones intermedias.

El CoT es especialmente interesante en prompts de tipo lógico o matemático.

- **Entrenamiento de un modelo de lenguaje para una tarea específica**

- Podemos entrenar a la IA de texto para que nos dé una respuesta más acorde con nuestras necesidades. Podemos hacerlo mediante el agregado de ejemplos etiquetados en un prompt con el objetivo de que el modelo de IA aprenda de ellos extrayendo patrones y características y consiga aportar de esta manera una respuesta más acorde a nuestras necesidades.

- **Zero shot prompt:** es un prompt sin muestras o ejemplos. Se usa cuando el modelo de IA puede realizar la tarea encomendada sin la necesidad de un entrenamiento.

- **One shot prompt:** prompt con una sola muestra o ejemplo. El único ejemplo en el prompt es para que la IA aprenda.

- **Few-shot prompt:** prompt con unas pocas muestras o ejemplos. Se añaden ejemplos etiquetados en el prompt para que la IA aprenda de ellos.

- **Recomendaciones para la creación de prompts**

- Evitar textos extensos: evita crear prompts que generen textos muy extensos ya que siempre hay más probabilidad de que el modelo cree repeticiones y bucles. Es preferible generar varios textos más pequeños donde la probabilidad de repeticiones y bucles es mucho menor.

- Escribe prompts que sean muy específicos y detallados: Esto evita las ambigüedades. Crea el prompt, revísalo, ejecútalo y luego modifícalo para ir optimizando. Cuanto más específico y detallado sea el prompt, menor será la probabilidad de que el modelo alucine.

- Separar el prompt en sus diferentes partes: Esto nos va a permitir analizar, revisar, modificar y reutilizar el prompt mucho más fácilmente.

- Visualiza los datos que el modelo no conoce y aportarlos en el prompt: Debes tener claro en cada prompt si debes aportar al modelo datos necesarios que este desconoce.

Chatbots conversacionales inteligentes de texto

- Los modelos de texto, basados en el procesamiento de lenguaje natural (PLN), pueden realizar tareas como resúmenes, mapas mentales y traducciones. Se entrenan para predecir la siguiente palabra en una secuencia, aprovechando patrones predecibles en el uso del lenguaje.

- Los grandes modelos de lenguaje (LLM) se entrenan con enormes conjuntos de datos de texto y usan redes neuronales profundas para procesar la información y aprender patrones en el lenguaje. Los GPT usan redes neuronales tipo transformer para el modelado de lenguaje en tareas específicas de PLN.

- Los transformadores generativos pre entrenados (GPT) son capaces de generar texto coherente de alta calidad, así como realizar una amplia variedad de tareas de PLN como resúmenes, mapas mentales, traducciones, etc.

- Los modelos conversacionales o de dialogo son modelos de generación de texto que están preparados para poder conversar.

- Los chatbots de IA generativa utilizan grandes modelos lingüísticos (LLM) para generar respuestas basadas en sus entradas. Se entrenan con conjuntos de datos masivos que contienen miles de millones de frases y oraciones. Aprovechan el aprendizaje profundo (basado en redes neuronales) y el PLN.

- **Etapas para crear un modelo de IA**

- **Etapa 1: preparar los datos.**

- Los datos que se usarán serán muchos datos y pueden combinar datos de código abierto y datos propios. Suelen considerarse grandes cantidades de texto de diferentes fuentes como libros, artículos, sitios web, etc.

- **Tareas:**

1. Se describen los datos (qué son);
2. Se filtran datos con los que no queremos que el modelo se entrene.
3. Se eliminan los datos duplicados.

- Como resultado de la etapa 1 obtenemos una pila de datos base no etiquetados y entrenamos el modelo de IA con ella.

- **Etapa 2: entrenar el modelo.**

1. Elegimos una arquitectura del modelo como Transformers.
2. Se empareja la pila de datos con ese modelo.
3. Tokenizamos la pila de datos.
4. Se inicia el proceso de entrenamiento sobre esos tokens. Este proceso puede llevar mucho tiempo, dependiendo del tamaño del modelo.
5. Durante el entrenamiento, el modelo aprende cómo los tokens se relacionan entre sí. No solo memoriza ejemplos específicos, sino que también generaliza a partir de ellos, permitiendo generar respuestas coherentes y contextualmente relevantes a nuevas preguntas o prompts.

- **Fase 3: Validación.**

Evaluar el modelo entrenado implica ejecutar el modelo y comprobar su rendimiento con respecto a un conjunto de puntos de referencia (benchmarks) que ayudan a determinar la calidad del modelo.

Para validar grandes modelos de lenguaje, se utilizan varias métricas especializadas que evalúan distintos aspectos del rendimiento del modelo. Por ejemplo:

- Medir la capacidad del modelo para predecir una muestra de texto.
- Medir la similitud entre el texto generado por el modelo y un texto de referencia.
- Medir el porcentaje de predicciones que coinciden con las respuestas de referencia.
- Medir la calidad de las traducciones automáticas.

- **Etapa 4: Puesta a punto.**

El desarrollador puede mejorar el rendimiento de un modelo pre-entrenado mediante instrucciones específicas y datos locales adicionales. Este ajuste fino permite adaptar el modelo a una tarea o dominio particular usando un conjunto de datos específico, y es más rápido que crear un modelo desde cero. La adaptación a nuevos datos mantiene el modelo actualizado con información reciente. La puesta a punto inicial se puede complementar con la adaptación continua a nuevos datos para mantener la relevancia del modelo a lo largo del tiempo.

- **Etapa 5: Despliegue del modelo:**

- El modelo puede desplegarse en una nube pública o integrarse en una aplicación o servicio existente.
- Se puede seguir mejorando el modelo con el tiempo con actualizaciones periódicas para ajustar el modelo con nuevos datos y mejorar su rendimiento.
- Aquí hay que preocuparse de que el sistema pueda manejar múltiples solicitudes simultáneas además de optimizar los recursos de computación para que la respuesta a solicitudes sea eficiente.

Arquitectura de los chatbot generativos inteligentes

o Interfaz del usuario: permite a los usuarios escribir preguntas o solicitudes en un cuadro de texto, donde el bot responde casi instantáneamente.

o Motor de PLN :

- Hace preprocesamiento de texto: divide texto en tokens, remueve stop words, maneja puntuación, y prepara el input para el modelo.
- Reconoce la intención del usuario detrás del prompt
- Hace reconocimiento de entidades: extrae las entidades claves del texto.

o Modelo grande de lenguaje: El LLM genera texto (crea texto basado en input, produciendo respuestas coherentes y contextualmente relevantes).

o Componente de gestión del diálogo: es crucial para mantener conversaciones coherentes y atractivas; además, se mantiene un flujo natural en las conversaciones.

o Integración con aplicaciones: en la integración con aplicaciones el modelo se accede a través de APIs desde aplicaciones o servicios.

La gestión del diálogo

• La gestión de diálogo involucra: mantener la pista de la historia de la conversación (incluyendo prompts y respuestas del sistema previos), gestión del estado del diálogo actual de la conversación (preferencias de usuario e interacciones previas), gestión de diálogo (mantiene la historia de diálogo, gestiona estrategias de conversación y decide las respuestas apropiadas), aprendizaje de políticas (crea caminos felices para las conversaciones, guiando interacciones hacia resultados positivos).

Diferencias entre un modelo lingüístico de propósito general y un chatbot conversacional inteligente

Un modelo lingüístico de propósito general (MLPG), como GPT-4 o BERT, es versátil y puede realizar diversas tareas, pero no está diseñado específicamente para interacciones conversacionales. Carece de gestión de diálogo, lo que significa que trata cada *prompt* de manera independiente, sin considerar el historial de interacciones. En contraste, una IA conversacional, como ChatGPT-4, está diseñada para mantener el contexto y la coherencia en las conversaciones. Esta IA reconoce las intenciones del usuario y puede seguir el estado de la conversación, lo que permite respuestas más humanas y atractivas. ChatGPT-4, por ejemplo, facilita intercambios dinámicos, haciendo preguntas aclaratorias y proporcionando información de seguimiento basada en las interacciones anteriores.

Arquitectura de transformers

• La arquitectura de un transformer permite el procesamiento paralelo de datos, siendo muy eficaz en tareas de procesamiento del lenguaje natural (NLP). Utiliza mecanismos de autoatención que ayudan al modelo a centrarse en diferentes partes del texto de entrada, mejorando así la comprensión del contexto y las relaciones entre palabras. Un transformer consta de un codificador, que procesa la entrada, y un decodificador, que genera la salida basándose en la representación vectorial del codificador. El modelo genera texto prediciendo una palabra a la vez, usando el contexto de las palabras anteriores.

El codificador

• Permiten procesar y entender secuencias de texto. Un codificador va a comprender el prompt porque a partir del texto de entrada generará representaciones contextuales que capturan el significado y las relaciones entre las palabras. El resultado es una serie de vectores que representan cada palabra en el contexto del prompt.

- Un codificador está compuesto por varias capas apiladas, cada una de las cuales realiza funciones específicas.

Fases del codificador

1. **Entrada de Texto:** El texto original es tokenizado y convertido en incrustaciones que representan matemáticamente cada palabra. Cada token se convierte a un vector numérico usando una técnica de incrustación (embedding).
2. **Aplicación de codificación posicional:** La técnica de codificación posicional agrega información sobre la posición de cada palabra en la secuencia, permitiendo al modelo entender el orden y las relaciones temporales entre las palabras.
3. **Cálculo de autoatención:** La capa de autoatención permite al modelo evaluar la relevancia de cada palabra en la entrada en relación con las demás. Utiliza un mecanismo que asigna diferentes pesos a las palabras según su importancia contextual. Estos pesos miden la importancia de cada token en relación con los otros en la secuencia, sumando 1 en total. Un peso alto entre dos tokens indica que su relación es crucial para entender el contexto en ese punto, mientras que un peso bajo sugiere que la relación es menos relevante.
4. **La capa de alimentación directa:** La capa de alimentación directa transforma la salida de la capa de autoatención utilizando una red neuronal que aplica funciones no lineales, lo que mejora la capacidad del modelo para aprender patrones complejos y genera una nueva representación para cada palabra, enriqueciendo su contexto. Esta capa principalmente refina las relaciones creadas por la autoatención, fortaleciendo la comprensión de dichas relaciones y facilitando la identificación de patrones más complejos. Este proceso se realiza a través de múltiples capas que combinan autoatención con alimentación directa.

- **Ejemplo:** consideremos la oración: La inteligencia artificial está revolucionando el futuro del trabajo humano rápidamente

➤ **Relaciones importantes en la auto-atención:**

- o "inteligencia artificial": Una relación directa y clave que establece el sujeto.
- o "revolucionando el futuro": Relación entre la acción y el objeto directo, que da contexto sobre el impacto.
- o "futuro del trabajo": Identifica específicamente el área afectada.
- o "trabajo humano": Añade un nivel adicional de especificidad sobre quién se ve afectado.

➤ **Refinamiento de relaciones en la alimentación directa:**

- o Profundización de "revolucionando el futuro": Captura la magnitud y la dirección del cambio, sugiriendo innovación y disrupción.
- o Especificación de "futuro del trabajo humano": Integra el concepto de transformación en el ámbito laboral, afectando a las habilidades y roles humanos.

• **Los codificadores también pueden ser utilizados en diversas tareas dentro del NLP:**

- o **Clasificación de Texto:** Determinar categorías o etiquetas para documentos.
- o **Reconocimiento de Entidades Nombradas:** Identificar nombres propios y otros términos relevantes dentro del texto.
- o **Análisis Sentimental:** Evaluar el tono emocional del contenido.

• **Los codificadores producen varias representaciones intermedias:**

- o **Vectorización:** Las palabras se transforman usando el mecanismo de embedding en vectores en un espacio multidimensional, donde las distancias entre los vectores reflejan similitudes semánticas.
- o **Representaciones Contextuales:** Cada palabra se representa no solo por su significado individual, sino también por su relación con otras palabras en la oración.
- o **Salidas para el Decodificador:** La salida final del codificador se pasa al decodificador, proporcionando un contexto rico para generar respuestas o traducciones.

El decodificador

• Los decodificadores en la arquitectura de los transformers son esenciales para generar salidas a partir de las representaciones contextuales de los codificadores. Se utilizan en diversas tareas de procesamiento del lenguaje natural (NLP), como la generación de texto, traducción automática y generación de respuestas a preguntas. Un decodificador está compuesto por varias capas apiladas, cada una con funciones específicas que contribuyen a la generación de salidas coherentes y relevantes.

Capas del decodificador

- Capa de autoatención: Permite que el decodificador preste atención a las palabras generadas previamente en la secuencia de salida. Crucial para mantener la coherencia y el contexto a medida que se genera texto.
- Capa de atención cruzada: Permite al decodificador acceder a la representación generada por el codificador. Utiliza la información contextual de la entrada para influir en la generación de la salida.
- Capa de alimentación directa: Transforma la salida de las capas anteriores utilizando una red neuronal completamente conectada.
- Codificación posicional: se utiliza una codificación posicional para permitir que el decodificador comprenda el orden de las palabras generadas.
- Capa lineal: toma la representación final del contexto generada por las capas anteriores y la transforma en una puntuación para cada palabra posible del vocabulario. Elige entre ellas la palabra de mayor puntuación.

Fases del decodificador

1. Entrada Inicial: La entrada consiste en los embeddings de las palabras generadas hasta ese momento, junto con información posicional.
 2. Cálculo de Autoatención: La capa de autoatención evalúa las palabras generadas hasta ahora para determinar su relevancia mutua.
 3. Atención Cruzada: Se realiza atención cruzada utilizando las representaciones del codificador para integrar información relevante del contexto original.
 - En otras palabras: combina la información contextual del codificador con la información de contexto del propio decodificador, creando así una salida coherente y contextualizada.
 4. Transformación a través de la Capa Feed-Forward: La salida se transforma mediante una red neuronal para obtener representaciones finales.
 5. Generación Final: La última capa del decodificador aplica una transformación lineal y una función softmax para producir probabilidades sobre el vocabulario, permitiendo seleccionar la palabra más probable como salida.
- La generación de la siguiente palabra en el decodificador ocurre en la capa final, después de que las capas de autoatención, atención cruzada y alimentación directa hayan procesado la información. Se obtiene una representación final del contexto hasta ese punto, que se pasa a una capa de salida (generalmente densa), transformándola en probabilidades para cada posible token siguiente. Esto resulta en una lista de palabras del vocabulario con sus respectivas probabilidades (denominada generación de logits), y se selecciona el token con la mayor probabilidad como la siguiente palabra.

- **Ejemplo:** si el decodificador está en el proceso de generar la siguiente palabra después de "El médico", podría producir una lista de palabras con probabilidades como:
 - "diagnosticó" (0.7)
 - "trató" (0.2)
 - "observó" (0.05)
 - "recetó" (0.05)
- En este caso, "diagnosticó" tendría la mayor probabilidad y sería seleccionada como la siguiente palabra en la secuencia.

Observaciones sobre los transformers

- La arquitectura de transformers se utiliza tanto para el entrenamiento del modelo como para procesar prompts y generar respuestas.
- Durante el entrenamiento, se utiliza la arquitectura de transformers para procesar grandes volúmenes de texto.

Entrenamiento de un transformer

- o Entrada: Se proporciona una oración completa, pero se oculta la última palabra.
- o Predicción: El modelo intenta predecir la palabra que falta basándose en el contexto de las palabras anteriores.
- o Comparación: La predicción del modelo se compara con la palabra real que se ocultó.
- o Cálculo de Error: Se calcula el error o la diferencia entre la predicción y la palabra real.
- o Retropropagación: Si la predicción es incorrecta, el error se utiliza para ajustar los pesos del modelo mediante un proceso llamado retropropagación. Este ajuste ayuda al modelo a mejorar sus predicciones futuras.
- Este ciclo se repite millones de veces con diferentes oraciones y datos, lo que permite que el modelo se vuelva más preciso y efectivo en sus predicciones.
- Otra forma de entrenar un transformer es usando un proceso llamado **enmascaramiento de lenguaje**. Consiste de las siguientes etapas:
 - o Enmascaramiento: Algunas palabras en la oración se reemplazan con un token especial.
 - o Predicción: El modelo intenta predecir las palabras que faltan basándose en el contexto de las palabras circundantes.
 - o Comparación: Se compara la predicción del modelo con las palabras reales.
 - o Cálculo de Error: El error se calcula y se utiliza para ajustar los pesos del modelo.
- Esta técnica permite que el modelo aprenda relaciones bidireccionales y entienda mejor el contexto completo de la oración.
- Los patrones aprendidos durante el entrenamiento son clave para que el modelo prediga la siguiente palabra en una secuencia. Al entrenarse con grandes volúmenes de texto, capta patrones en la estructura del lenguaje, las relaciones entre palabras y el contexto semántico, usando estos para hacer predicciones precisas. En lugar de almacenar estos patrones en una base de datos, están codificados en los millones de parámetros ajustados del modelo. Estos parámetros, afinados mediante comparaciones entre predicciones y respuestas correctas, permiten al modelo entender y generar texto coherente y contextualizado. Durante el entrenamiento, ajusta sus capas de autoatención y alimentación directa para aprender estos patrones.

Limitaciones de un bot conversacional de texto inteligente

Sesgos

Los modelos de IA pueden heredar sesgos de los datos con los que fueron entrenados. Estos pueden estar relacionados con género, raza, cultura, y más.

- Impacto: Las respuestas pueden reflejar y perpetuar prejuicios, llevando a resultados injustos o inapropiados.

Alucinaciones

Cuando el modelo genera información que es incorrecta o no basada en datos reales.

- Impacto: Puede llevar a la desinformación y a la pérdida de confianza en el modelo.

Repeticiones

Los modelos de IA pueden caer en patrones de repetición, especialmente si no reciben suficiente variabilidad en las entradas.

- Impacto: Las respuestas pueden volverse monótonas y parecer menos naturales o interesantes.

Capacidades Limitadas de Resolución de Problemas

Los bots pueden tener dificultades para resolver problemas complejos o no estructurados que requieren pensamiento crítico.

- Impacto: Esto puede resultar en la necesidad de intervención humana para resolver ciertos problemas.

Comprensión del Contexto Limitada

A veces, el bot puede tener dificultades para mantener el contexto de una conversación prolongada o compleja.

- Impacto: Esto puede llevar a respuestas fuera de contexto o irrelevantes.

Ambigüedad e Interpretación

Los bots pueden luchar con preguntas ambiguas o frases con múltiples significados.

- Impacto: Esto puede resultar en respuestas incorrectas o confusas para el usuario.

Dependencia de Entradas de Calidad

Los bots son tan buenos como los datos con los que han sido entrenados.

- Impacto: Datos sesgados o de baja calidad pueden llevar a respuestas sesgadas o inapropiadas.

EJERCICIO:

Asumir que se tiene un sistema de retorno de la información que usa el modelo de espacio vectorial que trabaja en base a una colección de archivos en una PC que no son páginas web, y que los términos de la consulta son palabras individuales. Una consulta es simplemente un AND de términos. La consulta retorna los documentos conteniendo al menos uno de los términos de la consulta, pero ordenados por su medida de relevancia.

1) Definir una función que devuelve la lista de los documentos resultantes de consulta sin ordenarlos por relevancia. Suponer que se usan índices invertidos.

2) ¿Qué propiedad conviene que cumpla la medida de relevancia de un documento?

Recordar, que para cada término de la consulta que está en el documento busca relevancia; entonces definir una función para calcular la medida de relevancia de un documento a partir de esas relevancias individuales.

3) Usando las dos funciones anteriores definir una expresión que da el resultado final del procesamiento de consultas.

1) obtener_documentos_sin_orden(inverted_index, consulta):

inverted_index: Diccionario con key= término y valor= cjo d docs que contienen ese término.

consulta: Lista de términos a buscar en los documentos.

return: Cjo d docs que contienen al menos uno de los términos de la consulta.

documentos_resultantes = set()

for termino in consulta:

if termino in inverted_index:

documentos_resultantes.add(inverted_index[termino])

```

return documentos_resultantes
2) Una métrica común es TF-IDF
calcular_relevancia(inverted_index, consulta, documento, total_docs):
    # param documento: Documento cuya relevancia se calculará.
    # total_docs: Número total de documentos en la colección.
    # return: Tupla (documento, relevancia).
    relevancia_total = 0
    for termino in consulta:
        if termino in inverted_index and documento in inverted_index[termino]:
            tf = inverted_index[termino][documento] # Frecuencia del término en el documento
            df = len(inverted_index[termino]) # Número de documentos que contienen el término
            idf = total_docs / df # Evita división por cero
            relevancia_total += tf * idf
    return (documento, relevancia_total)
3) procesar_consulta_con_relevancia(inverted_index, consulta, total_docs):
    # return: Lista de tuplas (documento, relevancia) ordenada por relevancia.
    documentos = obtener_documentos_sin_orden(inverted_index, consulta)
    documentos_relevancia = [ ]
    for doc in documentos:
        documentos_relevancia.add(calcular_relevancia (inverted_index, consulta,
        doc, total_docs))
    documentos_relevancia.mergesort-invert() #Para hacerlo en orden descendiente
    return documentos_relevancia

```

1) obtener_documentos_sin_orden

1. Recibir como entrada un **índice invertido**, que es un diccionario donde:
 - o La clave es un término (palabra).
 - o El valor es un conjunto de documentos en los que aparece ese término.
2. Recibir una **consulta**, que es una lista de términos (palabras clave).
3. Crear un conjunto vacío llamado **documentos_resultantes** para almacenar los documentos que contienen al menos uno de los términos.
4. Para cada término en la consulta:
 - o Si el término existe en el índice invertido, agregar todos los documentos asociados a ese término al conjunto de **documentos_resultantes**.
5. Devolver el conjunto final de documentos, que contiene los documentos relevantes para la consulta.

2) calcular_relevancia_documento_con_id

Calcular la relevancia de un documento basándose en la métrica TF-IDF

1. Recibir como entrada:
 - o Un **índice invertido**, donde cada término tiene un diccionario con documentos y sus frecuencias en ellos.
 - o Una **consulta**, que es una lista de términos a buscar en los documentos.
 - o El **documento** específico cuya relevancia se va a calcular.
 - o El **número total de documentos** en la colección.
2. Inicializar una variable **relevancia_total** con valor cero para acumular el puntaje del documento.
3. Para cada término en la consulta:
 - o Si el término está en el índice invertido y el documento contiene ese término:
 1. Obtener la frecuencia del término en el documento (TF).
 2. Contar cuántos documentos contienen el término (DF).

3. Calcular la puntuación de IDF usando la fórmula: $IDF = \frac{\text{total de documentos}}{DF}$
4. Multiplicar TF por IDF y sumar el resultado a **relevancia_total**.
4. Devolver una tupla con el identificador del documento y su puntuación de relevancia.

3) procesar_consulta_con_relevancia

1. Recibir como entrada:
 - Un **índice invertido**, donde cada término está asociado a documentos.
 - Una **consulta**, que es una lista de términos.
 - El **número total de documentos** en la colección.
2. Llamar a la función **obtener_documentos_sin_orden** para obtener la lista de documentos que contienen al menos uno de los términos de la consulta.
3. Crear una lista vacía para almacenar la relevancia de cada documento.
4. Para cada documento encontrado:
 - Calcular su relevancia llamando a la función **calcular_relevancia_documento_con_id**.
 - Guardar el documento junto con su puntuación en la lista.
5. Ordenar la lista de documentos en orden descendente según su puntuación de relevancia.
6. Devolver la lista de documentos ordenada, donde cada elemento es una tupla (**documento**, **relevancia**).